MAYAK

I built MAYAK for a construction company and still maintain it. It's where the team keeps project tasks, site photos, timesheets, material records and purchase requests. I handled the interface, application, database rules and deployment, then moved it from the original server to a VPS.

The stack is Next.js, React, TypeScript and self-hosted Supabase, running in Docker behind Nginx.

The job it needed to do

A request starts on a construction site, but several people in the office may need to act on it. The site manager knows what's needed. Procurement finds supplier options. Finance chooses the invoices. A dispatcher puts them in the payment register.

The useful thing to preserve is fairly simple: what was requested, what changed, who made the decision, and whose turn it is now. That became the basis of the procurement workflow.

The rest of the application

Tasks, photo reports, timesheets, materials, documents and contracts belong to a construction project. People see the records their company and project access allow. The modules share context, but approving an invoice does not automatically create a material receipt.

A task also has a review step: the assignee submits work, and the task author accepts or returns it. The manual status selector cannot skip acceptance. Management can resolve the review if the original author has been deleted.

About this case

The repository includes a 2:42 English walkthrough, diagrams and technical notes. The video uses invented data and an English reconstruction; it is not a production screen recording. Staff records, invoices and server credentials are excluded.

A request from start to finish

Take a request for plywood and concrete accessories. The site manager adds separate items, a required date and a priority, then sends it for approval. Once approved, procurement can attach supplier invoices under each item.

An invoice group has a title and comment and may contain several files. Finance chooses the actual files. Every requested item needs at least one selected invoice; the alternatives stay in the history.

What happens after finance

This was the part that needed care. Procurement still had an order to finish, while the dispatcher had invoices to register. I kept one request and gave registration its own linked entry and per-file marks. Procurement can finish independently; the register keeps completed entries visible.

Saving the decision

Finance approval is one database operation. The SQL routine locks the request row, checks the finance duty, active company and current stage, then rejects foreign files or a selection that leaves a material item uncovered.

It writes file decisions, group summaries, the next request status, the registry entry and an event in one transaction. A failed check leaves the approval uncommitted. Notifications happen separately.

There are shorter routes too

A direct invoice goes straight to finance and dispatch. Procurement can also finish a normal request without a finance stage when that is the actual outcome. Returns and recalls have their own checks and allowed destinations.

One regression left approved direct invoices in the ordinary working list, waiting for a procurement step they didn't have. I fixed that rule and added focused tests for both routes.

A few implementation details

Stale screens

Someone can leave a page open while a colleague moves the request. An ordinary transition includes the expected status in its update, so that old page can't overwrite the new stage. Its event write is separate; this is a different guarantee from the atomic finance routine.

Rank, duties and scope

Managerial rank controls authority. Supply, finance and dispatcher duties control department work, and can be combined without raising rank. Company and project rules constrain which records the person can access.

Next.js server actions use the user session to call Supabase through its HTTP gateway. PostgreSQL row-level security applies to ordinary queries. Some operations need privileged access; I keep those paths separate and check their actor and scope explicitly.

Downloads

The invoice handler authenticates the user, looks up the permitted file, fetches it through a short-lived Storage URL, and streams it with its original filename and private/no-store headers. This gave me one place to fix browser behaviour and the confusing filenames users had reported.

Phones and MAX

Photos are resized before upload. Small JPEGs are kept, and a failed conversion falls back to the original. The PWA has an offline fallback page; offline editing isn't implemented.

MAX pairing uses a random one-time code, stored as a hash. Messages link the next responsible person back to the request. Sending is best-effort, so MAYAK's working queue still matters when a notification fails.

Running it and fixing it

Moving to a VPS

The migration included database roles and grants, uploaded files, file attributes, environment configuration and HTTPS. A restored database wasn't enough on its own: the records could load while photographs and invoice downloads still failed.

Recovery

The current job keeps encrypted, versioned snapshots locally and offsite. I tested a cloud restore into a separate directory and database, including file attributes and proxy configuration. The password and recovery configuration have a separate off-server copy.

Checks and things left to improve

For this case, I reran 12 focused tests covering MAX codes, direct-invoice list behaviour and activity-report filtering/export. All passed. SQL regression scripts and a backup integration test also exist; this is not a complete browser end-to-end suite.

There are still things I'd improve. The service runs on one VPS. Notifications have no durable outbox. Material balances are calculated from movement history, without published large-scale performance measurements. Live files and the database aren't one atomic snapshot.

My work

I worked across requirements, interface design, the application and database, integrations, deployment and production fixes. Feedback led to changes such as file-level invoice selection, a separate register lifecycle, simpler mobile controls and original download filenames.

I haven't measured an attributable ROI or percentage time saving. What I can show is the deployed product, the rules it implements, the fixes made from use, and a recovery procedure I've exercised.